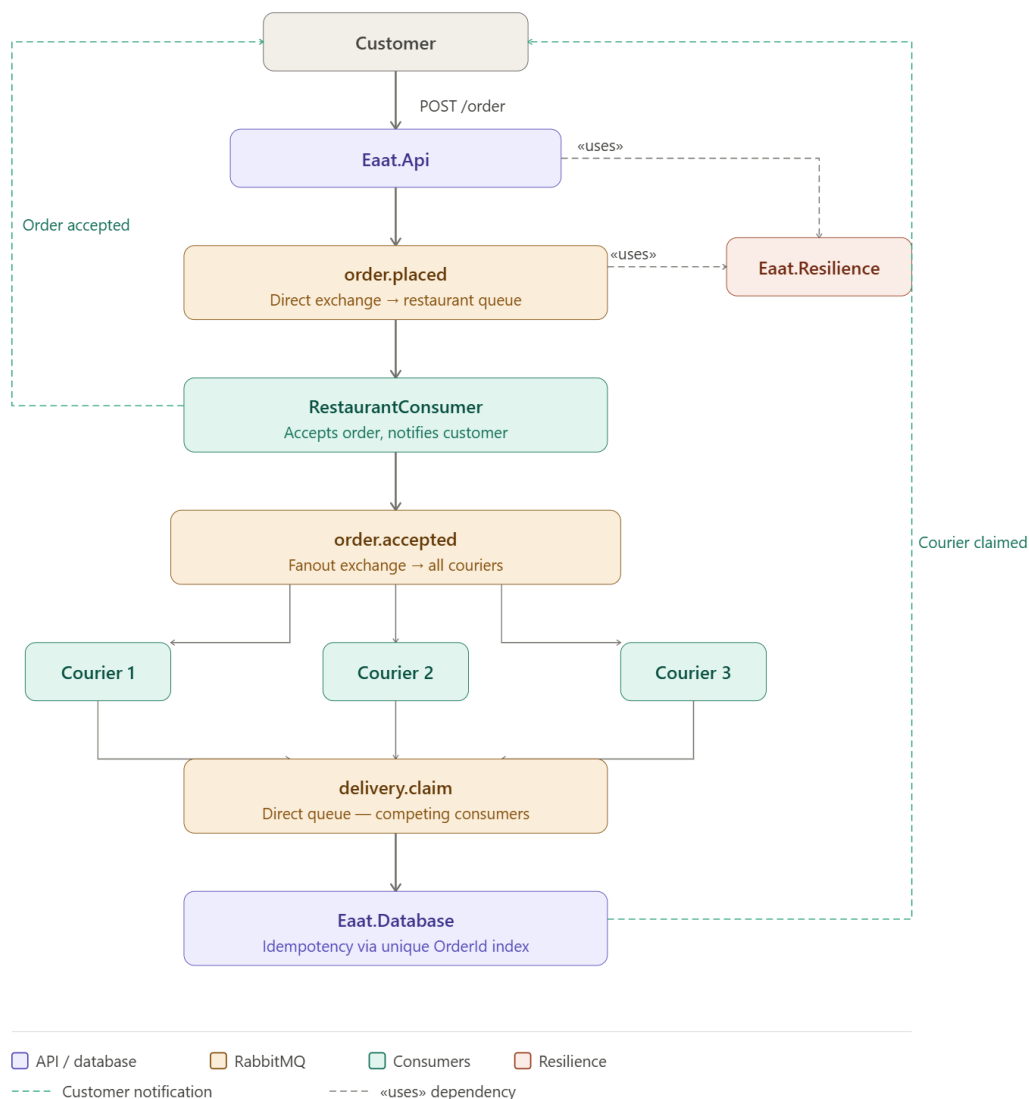




Her ses et flowchart af EAAT-systemet. Diagrammet viser kun én restaurant af hensyn til overskuelighed, men systemet er designet til at kunne håndtere flere restauranter. Dette uddybes nærmere i resten af rapporten.

Systemet er bygget med modularitet i tankerne og består af følgende komponenter: Eaat.Api, Eaat.Models, Eaat.RabbitMQService, Eaat.RestaurantService, Eaat.CourierService, Eaat.Resilience og Eaat.Database.

Pub-Sub bruges til at håndtere beskedafsendelse på tværs af systemet. Når en ordre placeres, publiceres en besked til RabbitMQ, som distribuerer den til den relevante restaurant vha. deres restaurantID. Når restauranten accepterer, publiceres en ny besked som distribueres til alle kurérerne.

Competing Consumers og Idempotency bruges i sammenspil til at håndtere "første til mølle" princippet. Når en ordre er accepteret, konkurrerer alle kurérere om at "claime" leveringen ved at lytte på den samme kø. Første til mølle implementeres ved at den første kurér der inserter i databasen vinder.

Dette garanteres af et UNIQUE index på OrderId. De resterende kurérere rammer index constraint og får en DbUpdateException (Denne fejl ses som en accepteret error, og ignoreres i logging), som systemet håndterer som et tabt race. Uanset hvor mange kurérere der forsøger, vil resultatet altid være det samme: én kurér får opgaven.

Da constrainten håndhæves på databaseniveau opnås atomicitet, hvilket gør implementationen robust overfor race conditions i et concurrent system og understøtter dermed scalability.

Outbox Pattern bruges til at opnå atomicitet ved afsendelse af ordrebeskeder.

Der er implementeret en background worker, som hvert 5. sekund kontrollerer databasen for ubehandlede outbox-beskeder. Hvis der findes beskeder, publiceres de til RabbitMQ og markeres derefter som behandlede.

Dette sikrer, at hvis systemet går ned under behandling af en ordre, vil database-transaktionen stadig garantere, at både ordren og den tilhørende outbox-besked enten gemmes fuldstændigt eller rulles tilbage. Ved genstart vil polling-processoren fortsætte behandlingen af de beskeder, som endnu ikke er markeret som behandlede.

Dermed undgås situationer, hvor en ordre eksisterer i databasen uden at den tilhørende besked er blevet distribueret til de øvrige services.

Data Konsistens opnås gennem tre mekanismer.

Eventual consistency sikrer at alle komponenter til sidst når samme tilstand, selvom de opdateres asynkront. Message-based consistency betyder at ingen services kalder hinanden direkte — al kommunikation sker via beskeder fra API'en.

Med autoAck: false sikres at beskeder kun fjernes fra køen når de er behandlet succesfuldt og dermed opnås at-least-once-delivery. Dette betyder at beskeder kan blive leveret mere end én gang i tilfælde af fejl, hvilket gør idempotency nødvendigt for at undgå duplicate handlinger.

Notifikationer til kunden

Notifikationer til kunden sker løbende. Først da restauranten accepterer ordren, her nævnes det at maden bliver forberedt. Når kuréren så henter ordren, notificeres kunden også om at deres ordre bliver leveret. Med dette flow opnår vi en klar kæde af events igennem systemet, som i sidste ende leder tilbage til kunden som startede kæden (I selve systemet er dette blot en konsol udskrivning da dette blot er prototypisk).

Hvorfor bruge en direct queue per restaurant?

Dette gøres, for at lade kunden vælge den specifikke restaurant de ønsker mad fra. Restauranterne kunne med fordel for kunden inddeles i kategorier.

Hvorfor bruge fanout til kurérene?

Da alle kurérer skal have lige mulighed for at samle ordren op, skal de alle notificeres om at en ordre eksisterer. Dette lade fanout exchange-typen os gøre.

Hvorfor RabbitMQ over HttpClient?

Systemet er bygget på en event-driven arkitektur, hvor hver komponent reagerer på events frem for at kalde hinanden direkte. Dette løskobler systemet og gør det nemt at tilføje nye komponenter uden at ændre eksisterende kode.

RabbitMQ håndterer routing gennem exchanges, bufferer beskeder, dvs. holder på dem indtil modtageren er tilgængelig, ved nedetid og sikrer at systemet kan fungere selvom enkelte services midlertidigt er utilgængelige.

Fejlhåndtering og resilience

Dead-letter-queue (hypotetisk, ikke implementeret) kan anvendes til at håndtere fejlbehæftede beskeder i systemet. Hvis en besked fejler gentagne gange under behandling, vil den blive flyttet til en separat kø. Denne kø kan overvåges af udviklere, hvilket gør det muligt at identificere og rette skjulte fejl uden at påvirke det primære flow i systemet.

En besked kunne eksempelvis havne i DLQ hvis restauranten ikke kan deserialisere beskeden, eller hvis RabbitMQ ikke kan levere beskeden efter maksimalt antal forsøg.

Polly's Resilience Pipeline anvendes til implementering af de centrale resilience-strategier Retry, Circuit Breaker og Timeout. Disse strategier sikrer, at systemet kan håndtere midlertidige fejl og ustabile afhængigheder på en kontrolleret måde.

Retry anvendes til at forsøge en operation igen, hvis den fejler på grund af midlertidige problemer såsom netværksfejl eller kortvarig utilgængelighed af en service. Dette øger sandsynligheden for, at operationen gennemføres succesfuldt uden manuel indgriben.

Circuit Breaker anvendes til at beskytte systemet mod gentagne fejl mod den samme afhængighed. Hvis en operation fejler kontinuerligt, åbnes kredsløbet midlertidigt, hvilket stopper yderligere forsøg i en periode. Dette forhindrer overbelastning og reducerer risikoen for kaskadefejl i systemet.

Timeout sikrer, at operationer ikke hænger uendeligt. Hvis en ordre ikke kan behandles inden for en defineret tidsgrænse, f.eks. 5 sekunder, afbrydes processen. Dette frigiver ressourcer og sikrer, at systemet forbliver responsivt for brugeren.

Hvordan opnås scalability?

Scalability opnås gennem systemets event-drevne arkitektur og brugen af RabbitMQ som message broker. Hver komponent kan skaleres uafhængigt ved at tilføje flere instanser, uden at påvirke resten af systemet.

Systemet er bygget med modularitet i tankerne. De enkelte komponenter er adskilt i separate class libraries med klare ansvarsområder, hvilket gør det nemt at vedligeholde og udvide systemet. Arkitekturen er desuden klar til en eventuel migration til egentlige microservices, da kommunikationen allerede foregår via RabbitMQ frem for direkte kald mellem komponenterne.